ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ №1

Дисципліна: Основи програмної інженерії | Тема: Збір і аналіз інформації про предметну область

1. Загальна інформація про проєкт

Найменування проєкту: Budgettracker (Budget & Expense Tracker)

Опис проєкту: Budgettracker — це вебплатформа та інтегрований застосунок для ведення персонального та групового фінансового обліку (Budget & Expense Tracker). Система дозволяє користувачам ефективно відстежувати доходи та витрати, здійснювати категоризацію транзакцій, формувати наочні звіти й аналітику за обрані періоди, а також планувати бюджет. Проєкт орієнтований на студентів та молодих спеціалістів для покращення їхньої фінансової грамотності та зручного контролю витрат.

2. Склад команди та розподіл ролей

Учасник	Роль у проєкті	Функції та зона відповідальності
Учасник 1	Project Manager / System Analyst	Координація розробки, визначення вимог, збір інформації про предметну область, ведення документації проєкту.
Учасник 2	Frontend / Full-stack Developer	Розробка користувацького інтерфейсу (UI/UX) на React, інтеграція з REST API, побудова графіків та звітів.
Учасник 3	Backend Developer / DB Architect	Проєктування бази даних (PostgreSQL / SQLite), розробка серверної логіки (Express/Node.js), налаштування авторизації та API.

3. Звіт про вибір задачі

3.1. Назва та призначення системи

Budgettracker — Budget & Expense Tracker. Автоматизація обліку особистих фінансів, візуалізація витрат за категоріями, планування щомісячних лімітів та аналіз фінансових звичок користувача.

3.2. Порівняння з аналогами та ефект від впровадження

На відміну від аналогів (Monefy, Spendee, Wallet), Budgettracker надає простий вебінтерфейс без надлишкового функціоналу та платних підписок. Ефект полягає у забезпеченні молоді зручним інструментом контролю витрат та оптимізації бюджету.

3.3. Мотивація та аналіз здійсненності

Проєкт є актуальним через необхідність управління обмеженим бюджетом у студентські роки. Для реалізації використовуються ПК розробників, Node.js, React, VS Code та PostgreSQL. Основний ризик — недотримання дедлайнів, що мінімізується чітким скрам-контролем.

3.4. Модель виробничого циклу

Обрано гнучку методологію **Agile / Scrum**. Вона дозволяє розбивати процес розробки на короткі спринти, швидко отримувати зворотний зв'язок, коригувати вимоги та забезпечує паралельну розробку Frontend і Backend модулів.

4. Відповіді на контрольні запитання

1. Основні методи збору інформації: Опитування та анкетування користувачів, інтерв'ю з експертами, спостереження за робочими процесами, аналіз документації та продуктів-конкурентів.

2. Інструменти для збору даних: Google Forms, сервіси опитувань, ментальні карти (Miro, MindMeister), таск-трекери (Jira, Trello) та аналітичні інструменти.

3. Важливість аналізу до початку проєкту: Дозволяє чітко сформулювати вимоги, уникнути непорозумінь із замовником, оптимізувати витрати ресурсів та запобігти переробкам архітектури.

4. Основні етапи аналізу: 1) Визначення мети та меж; 2) Збір первинних даних; 3) Систематизація та структурування; 4) Виявлення проблем; 5) Формулювання функціональних вимог.

5. Вибір релевантних джерел: Використання авторитетних джерел, пряме опитування цільової аудиторії, аналіз профільних досліджень та аналогів ринку.

6. Оцінка достовірності та якості даних: Перехресна перевірка даних (триангуляція), експертна оцінка, статистичний аналіз результатів та тестування гіпотез.

7. Забезпечення повноти та об'єктивності: Залучення різних груп користувачів, уникнення суб'єктивних припущень та систематичне документування вимог.

8. Виявлення ключових аспектів та проблем: Побудова діаграм прецедентів (Use Case), карти шляху користувача (User Journey Map) та аналіз зворотного зв'язку.

9. Врахування потреб користувачів: Створення персонажів (User Personas), сценаріїв використання (User Stories) та пріоритезація за методологією MoSCoW.

10. Формулювання вимог на основі аналізу: Трансформація виявлених потреб у чіткі, вимірювані та несуперечливі функціональні та нефункціональні вимоги в специфікації (SRS).